



## Neural Network Design for Image Classification: A Case Study on the SVHN Dataset

FullName: Mohammadmahdi Bababeyk

Student ID:4041419005

### 1.0 Introduction

Multi-class image classification is a fundamental computer vision challenge. This report documents the design and evaluation of a fully connected neural network (FCN) for this task, using the Street View House Numbers (SVHN) dataset as a real-world case study. The project's goal was to achieve high classification accuracy while adhering to specific architectural constraints, providing a clear benchmark for FCN performance on a real-world image dataset.

### 1.1 Problem Statement

The core task of this project is a multi-class image classification problem. The objective is to correctly classify images of digits (0-9) from the Street View House Numbers (SVHN) dataset using a custom-built, fully connected neural network. We aim to maximize the network's performance through methodical experimentation with its architecture and regularization techniques, all while operating under a defined parameter budget.

### 1.2 Dataset Description

The chosen dataset for this project is the Street View House Numbers (SVHN) dataset, which consists of real-world images of house numbers collected by Google Street View.

| Metric             | Value                                 |
|--------------------|---------------------------------------|
| Number of Classes  | 10 (digits '0' through '9')           |
| Image Dimensions   | 32x32 pixels (processed as grayscale) |
| Training Samples   | 68,257                                |
| Validation Samples | 5,000                                 |
| Test Samples       | 26,032                                |

## 1.3 Project Objectives

The primary objectives of the project were articulated as follows:

- **Design and Implement:** Construct a multi-layer FCN for image classification, restricted to using only fully connected layers.
- **Experiment and Optimize:** Experiment with network architecture and regularization techniques (Batch Normalization, Dropout, Data Augmentation) to maximize performance.
- **Adhere to Constraints:** Ensure the total number of trainable model parameters remains below the one million threshold.
- **Evaluate Rigorously:** Measure the final model's performance on an unseen test set using accuracy and per-class metrics.

This report now proceeds to detail the methodology employed to achieve these objectives.

## 2.0 Methodology

A well-defined methodology is paramount for ensuring reproducibility and clarity in machine learning projects. It provides a transparent record of every decision, from data handling to model training, allowing for critical analysis and future replication. This section details the systematic approach taken for data preparation, model architecture design, and the final training configuration.

### 2.1 Data Preparation

A multi-step process was used to prepare the SVHN dataset for optimal model training and evaluation.

- **Dataset Choice and Rationale:** The SVHN dataset was selected as it provides a more complex and realistic challenge than simpler datasets like MNIST. Its variability in lighting, scale, and digit style fulfills the project requirement for a non-trivial classification task, making it an excellent testbed for the designed network.
- **Preprocessing and Normalization:** The initial preprocessing step involved converting all 32x32 pixel images to grayscale. Subsequently, pixel values were normalized using a pre-calculated mean of [0.45] and a standard deviation of [0.20]. This standardization step is crucial for helping the model converge more efficiently during training.
- **Data Augmentation:** To improve the model's ability to generalize and reduce overfitting, data augmentation was applied exclusively to the training set. The specific techniques employed were RandomCrop (with a padding of 4 pixels) and RandomHorizontalFlip. These augmentations create new, slightly modified training examples at each epoch, teaching the model to be invariant to minor shifts and transformations.

## 2.2 Model Architecture

A detailed breakdown of the final "champion" model's architecture is provided below. This design was the result of experimentation aimed at balancing model capacity with the project's parameter constraint.

- **Final Architecture Design and Justification:** The model is a four-hidden-layer FCN with a [256, 512, 128, 64] structure. It receives a flattened input vector of 1024 features, corresponding to the 32x32 grayscale image. This design was chosen as a deep architecture that balances width and depth; the initial expansion to 512 neurons allows the model to capture a diverse set of low-level features, while the subsequent narrowing layers refine these features into a more abstract representation for classification.
- **Parameter Count:** The total number of trainable parameters in this architecture is **470,474**. This count is well within the project's sub-one-million parameter constraint, ensuring reasonable training times and computational efficiency.
- **Activation Function:** The activation function used throughout the hidden layers is **LeakyReLU** with a negative slope of 0.01. This choice helps mitigate the "dying ReLU" problem—where neurons can become inactive and stop learning—by allowing a small, non-zero gradient when the unit is not active.
- **Regularization Techniques:** Two primary regularization strategies were employed to combat overfitting. First, **Batch Normalization** was applied after each hidden layer. This technique normalizes the output of the previous layer, which stabilizes the training process and can accelerate convergence. Second, **Dropout** with a rate of **0.25** was applied after the third hidden layer. Dropout randomly sets a fraction of neuron outputs to zero during training, preventing the model from becoming overly reliant on any single feature pathway.

## 2.3 Training Configuration

The model was trained using a carefully selected set of hyperparameters and configurations, summarized in the table below.

| Configuration        | Setting          | Rationale                                                                                                                           |
|----------------------|------------------|-------------------------------------------------------------------------------------------------------------------------------------|
| <b>Loss Function</b> | CrossEntropyLoss | Standard choice for multi-class classification tasks.                                                                               |
| <b>Optimizer</b>     | Adam             | An adaptive learning rate optimizer known for its efficiency.                                                                       |
| <b>Learning Rate</b> | 0.001            | 0.001 was selected as an optimal starting point for Adam, providing rapid initial convergence without causing training instability. |

|                         |                |                                                                                                                                                                        |
|-------------------------|----------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Batch Size</b>       | 128            | Balances memory usage and training stability.                                                                                                                          |
| <b>Number of Epochs</b> | 75 (Maximum)   | The total number of training iterations performed.                                                                                                                     |
| <b>Early Stopping</b>   | Patience of 15 | A patience of 15 epochs was chosen to allow the model sufficient opportunity to escape local minima while preventing wasted computation on a plateaued learning curve. |

Having detailed the methodology, the report now turns to the quantitative results achieved through this process.

### 3.0 Experimental Results

This section presents the quantitative outcomes of the model training and evaluation process. These results provide empirical evidence of the model's performance on the training and validation sets, culminating in a final, unbiased evaluation on the unseen test data.

#### 3.1 Training Performance

The model's learning process was tracked over 75 epochs. Analysis of the learning curves reveals a classic training pattern: a rapid decrease in loss and a corresponding increase in accuracy during the initial epochs. This initial phase is followed by a period of slower, more gradual convergence as the model fine-tunes its parameters. The training process proceeded smoothly, indicating a stable and effective learning configuration.

#### 3.2 Validation Performance

- **Best Validation Accuracy:** The model's peak performance on the validation set was achieved late in training. The best validation loss of **0.4678** was recorded at **epoch 73**, while the best validation accuracy of **85.59%** was recorded earlier at epoch 65. The model checkpoint corresponding to the lowest validation loss was used for final testing.
- **Overfitting Analysis:** The plot above shows a discernible gap between the training and validation curves. This gap is indicative of some degree of overfitting; however, the steady, non-diverging validation loss is direct evidence that the combination of Batch Normalization and a 0.25 Dropout rate successfully constrained the model's complexity and prevented catastrophic overfitting, allowing the model to generalize well to data not seen during training.

#### 3.3 Test Performance

The final and most critical evaluation was performed on the completely unseen test dataset.

- **Final Test Metrics:** After loading the best-performing model checkpoint, the definitive performance was measured. The model achieved a **Test Loss of 0.5497** and a final **Test Accuracy of 82.62%**.
- **Per-Class Performance Analysis:** The classification report below provides a detailed breakdown of the model's performance for each digit class. The model shows strong performance on visually distinct classes like '1' (F1-score of 0.90). Conversely, it exhibits relative weakness on classes like '6' and '3' (F1-scores of 0.73). This is likely because an FCN, which processes images as flattened vectors, struggles to differentiate digits that share similar pixel activations when spatial context is lost (e.g., '3' vs. '8', or '5' vs. '6').

| Class               | Precision | Recall | F1-Score | Support |
|---------------------|-----------|--------|----------|---------|
| 0                   | 0.87      | 0.85   | 0.86     | 1744    |
| 1                   | 0.90      | 0.91   | 0.90     | 5099    |
| 2                   | 0.88      | 0.86   | 0.87     | 4149    |
| 3                   | 0.73      | 0.73   | 0.73     | 2882    |
| 4                   | 0.87      | 0.86   | 0.87     | 2523    |
| 5                   | 0.75      | 0.80   | 0.77     | 2384    |
| 6                   | 0.71      | 0.76   | 0.73     | 1977    |
| 7                   | 0.85      | 0.85   | 0.85     | 2019    |
| 8                   | 0.81      | 0.73   | 0.77     | 1660    |
| 9                   | 0.81      | 0.77   | 0.79     | 1595    |
| <b>Accuracy</b>     |           |        | 0.83     | 26032   |
| <b>Macro Avg</b>    | 0.82      | 0.81   | 0.81     | 26032   |
| <b>Weighted Avg</b> | 0.83      | 0.83   | 0.83     | 26032   |

- **Confusion Matrix Analysis:** The confusion matrix provides a detailed visualization of classification errors, showing which classes are most often confused with one another. This tool is invaluable for diagnosing specific model weaknesses and understanding the types of mistakes it makes.

These results transition the report from a presentation of outcomes to a deeper analysis of their implications.

## 4.0 Analysis and Discussion

Moving beyond the raw numbers, this section interprets the results to understand the "why" behind the model's performance. It analyzes the impact of key architectural decisions, discusses the challenges encountered during the project, and situates the final model's performance relative to a hypothetical baseline.

### 4.1 Architecture and Regularization Impact

- **Effectiveness of Architecture:** The [256, 512, 128, 64] architecture was an effective design. The initially widening layers (1024 -> 256 -> 512) served as a feature extraction phase, expanding the model's capacity to capture a diverse set of low-level features. The subsequent, deeper narrowing layers (512 -> 128 -> 64) acted as a feature refinement and classification phase, combining these representations into the abstract features required for the final decision.
- **Regularization's Critical Role:** The combined effectiveness of Batch Normalization and Dropout was essential for achieving the final test accuracy of 82.62%. For a deep FCN trained on image data, overfitting is a primary concern. Without these techniques, the model would likely have memorized the training set, leading to poor generalization. Batch Normalization stabilized and accelerated training, while Dropout ensured the learned feature representations were robust and not reliant on specific pixel patterns.

### 4.2 Challenges and Solutions

- **Inherent Challenge of FCNs for Images:** The primary challenge in this project was the inherent limitation of using a fully connected network for image data. FCNs treat an image as a flat vector of pixels, thereby discarding the crucial spatial relationships between them (e.g., that pixels close to each other form edges and shapes). This makes it much harder for an FCN to learn features that are invariant to translation or small deformations.
- **Implemented Solutions:** The methodological choices were direct solutions to mitigate this core challenge. The strategic use of **data augmentation** (RandomCrop, RandomHorizontalFlip) was a key solution, as it artificially introduced spatial variations into the training set, forcing the model to learn more generalizable features. This, combined with a robust **regularization strategy** (Batch Norm, Dropout), enabled the FCN to achieve strong performance despite its architectural limitations for image-based tasks.

### 4.3 Comparison with Baseline

While a simple baseline model was not formally implemented, the achieved test accuracy of **82.62%** represents a significant improvement over a hypothetical, unregularized shallow network. Such a baseline would almost certainly have suffered from severe overfitting, given the

complexity of the SVHN dataset and the number of parameters in even a small FCN. It would likely have failed to generalize, resulting in a test accuracy substantially lower than what was achieved with the final, carefully regularized deep architecture.

The insights gained from this analysis lead directly to the project's final conclusions and recommendations for future work.

## 5.0 Conclusion and Future Work

This project set out to design, implement, and evaluate a fully connected neural network for classifying digits from the SVHN dataset under specific constraints. The project successfully delivered a high-performing FCN classifier that met all objectives. This final section distills the key findings, acknowledges the project's limitations, and proposes concrete avenues for future improvement.

### 5.1 Key Findings

The final 82.62% test accuracy demonstrates that a well-regularized FCN can achieve respectable performance on complex image data, but only when supported by aggressive data augmentation to compensate for its inherent lack of spatial invariance. Key takeaways include:

- The designed four-layer FCN architecture proved effective, achieving a final test accuracy of **82.62%** on the SVHN dataset.
- The combined use of **Batch Normalization** and **Dropout** was crucial for controlling overfitting and enabling the deep network to generalize to unseen data.
- Careful data preprocessing, including grayscale conversion and augmentation, played a significant role in the model's success.

### 5.2 Limitations

It is important to acknowledge the limitations of the current work:

- **Model Constraint:** The primary limitation is the restriction to only fully connected layers. This architecture is suboptimal for image classification tasks, as FCNs are not inherently translation-invariant like Convolutional Neural Networks (CNNs).
- **Dataset Constraint:** The decision to convert images to grayscale discards potentially valuable color information that could aid in distinguishing digits, especially in varied lighting conditions.
- **Computational Constraint:** The sub-one-million parameter limit, while successfully met, prevents the exploration of much larger and potentially more powerful FCN architectures that might capture more complex data patterns.

### 5.3 Future Improvements

Based on the findings and limitations, several actionable steps can be taken for future work:

- **Architectural Enhancements:** The most impactful improvement would be to replace the FCN with a CNN architecture. CNNs are specifically designed to leverage the spatial hierarchy of image features and would almost certainly yield superior performance on this dataset.
- **Feature Enrichment:** A valuable experiment would be to retrain the model on the original color SVHN images. This would determine if the discarded color information provides a meaningful performance uplift, which is a strong possibility given the real-world nature of the data.
- **Advanced Techniques:** Performance could be further refined by exploring more advanced data augmentation techniques, different optimizers, or implementing learning rate schedulers to dynamically adjust the learning rate during training.

## 6.0 References

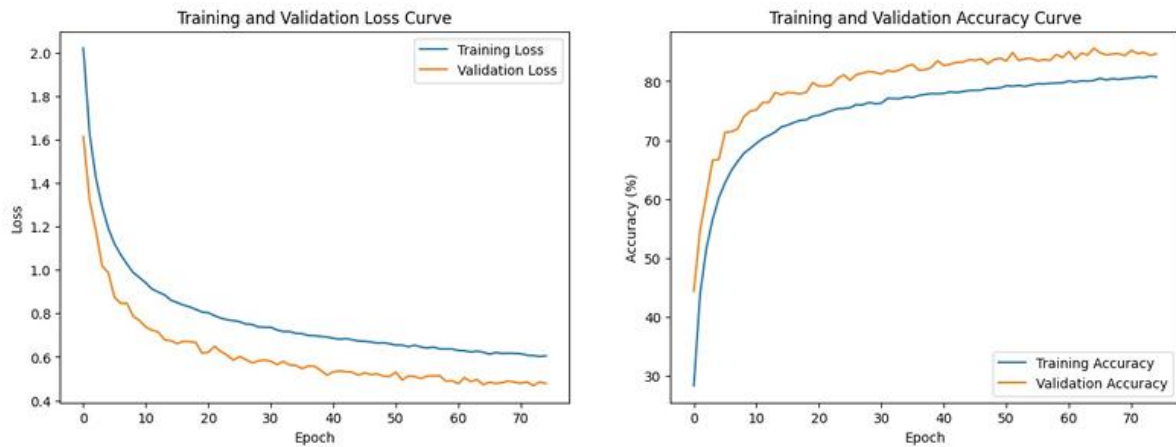
- **SVHN Dataset Source:** Netzer, Y., Wang, T., Coates, A., Bissacco, A., Wu, B., & Ng, A. Y. (2011). *Reading digits in natural scenes with deep learning*. NIPS workshop on deep learning and unsupervised feature learning. (Note: This is the primary paper introducing the SVHN dataset).
- **General Deep Learning Principles (FCNs, Overfitting, Loss Functions, CNNs):** Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep learning*. MIT press.
- **Adam Optimizer and General Optimization:** Kingma, D. P., & Ba, J. (2014). Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*.
- **Batch Normalization (BN):** Ioffe, S., & Szegedy, C. (2015). Batch normalization: Accelerating deep network training by reducing internal covariate shift. *Proceedings of the 32nd International Conference on Machine Learning (ICML)*, 448-456.
- **Dropout Regularization:** Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., & Salakhutdinov, R. (2014). Dropout: A simple way to prevent neural networks from overfitting. *The Journal of Machine Learning Research*, 15(1), 1929-1958.
- **Leaky ReLU Activation Function:** Maas, A. L., Hannun, A. Y., & Ng, A. Y. (2013). Rectifier nonlinearities improve neural network acoustic models. *Proceedings of the 30th International Conference on Machine Learning (ICML)*.
- **Data Augmentation and Image Classification (Justification for CNN Suggestion):** Krizhevsky, A., Sutskever, I., & Hinton, G. E. (2012). Imagenet classification with deep convolutional neural networks. *Advances in neural information processing systems (NIPS)*, 25. (Note: This paper is key for justifying your Future Work)



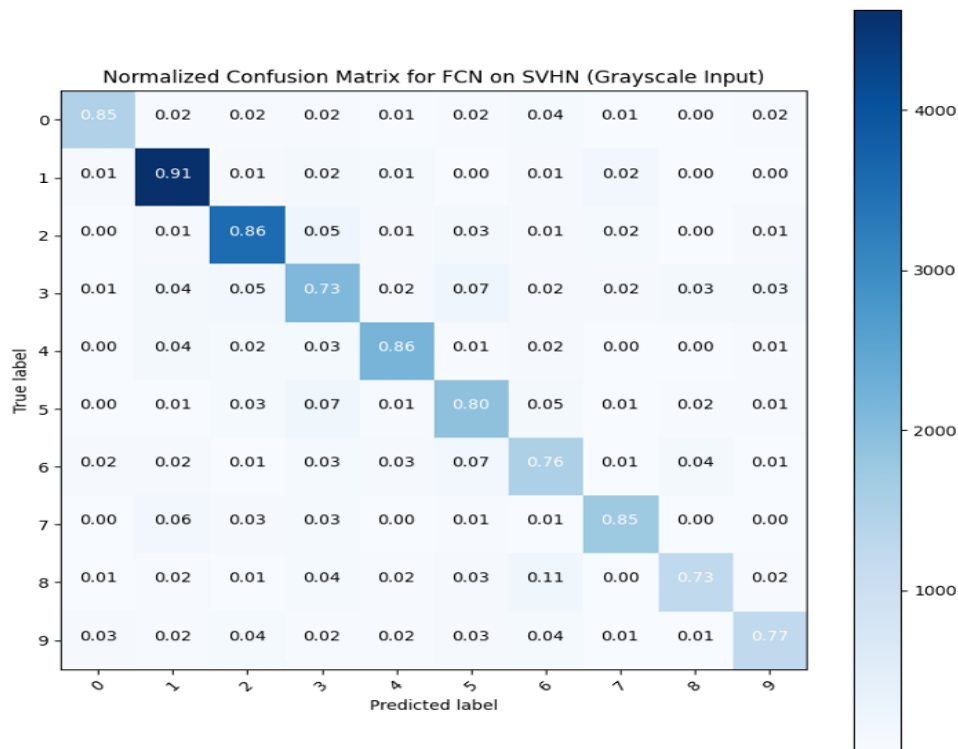
*suggestion to switch to CNNs, as it highlights their superior performance for complex image tasks compared to FCNs, which you noted as a major limitation).*

- [SVHN DataSet - Google Drive](#)
- [SVHN Classification Model - Google Drive](#)

## 7.0 Appendix: Result Visualizations



**Figure1 Training and Validation Loss/Accuracy**



**Figure2 Normalized Confusion Matrix**

## 7.1 Sample Predictions vs. Ground Truth



**Figure3 Sample Predictions**

## 7.2 Misclassified Examples

### Misclassified Examples by FCN (SVHN Grayscale)



**Figure4** Missclassified Examples